

EXPERIMENT 28:

28. Consider a file system where the records of the file are stored one after another both physically and logically. A record of the file can only be accessed by reading all the previous records. Design a C program to simulate the file allocation strategy.

AIM:

To design a C program to simulate the Sequential File Allocation strategy where records are stored contiguously.

The screenshot shows a C program in a code editor with a dark theme. The code is in a file named 'main.cpp'. It includes a 'Run' button. The code defines an array 'disk' of size 100, initialized to 0. It prompts the user to enter the total blocks on disk, then the starting block address and the length of the file. It then checks if the file can be allocated contiguously. If yes, it prints the allocation details. If no, it prints an error message. The output window shows the execution results for two test cases.

```
main.cpp Run Output
3 int main() {
4     int disk[100] = {0};
5     int total_blocks, start_block, length, choice;
7     printf("Enter total blocks on disk: ");
8     scanf("%d", &total_blocks);

    } while (choice
== 1);
42
43
44     return 0;

Output
Enter total blocks on disk: 20
Enter starting block address: 2
Enter length of file (number of blocks): 4
File allocated contiguously from block 2 to 5.
Do you want to add another file? (1 for Yes / 0 for No): 1
Enter starting block address: 3
Enter length of file (number of blocks): 3
Allocation failed: Contiguous blocks are already occupied.
Do you want to add another file? (1 for Yes / 0 for No): 0
=== Code Execution Successful ===
```

PSEUDO CODE :

BEGIN

PRINT "Enter total blocks on disk:"

READ total_blocks

DECLARE disk array of size total_blocks INITIALIZED to 0

LOOP

PRINT "Enter file starting block and length (number of blocks):"

READ start_block, length

SET is_allocated = 1

```

// Check if start bounds and contiguous blocks are free
IF start_block + length > total_blocks THEN
    PRINT "Error: Out of disk bounds!"
    SET is_allocated = 0
ELSE
    FOR i FROM start_block TO (start_block + length - 1) DO
        IF disk[i] == 1 THEN
            SET is_allocated = 0
            BREAK
        END IF
    END FOR
END IF

IF is_allocated == 1 THEN
    FOR i FROM start_block TO (start_block + length - 1) DO
        SET disk[i] = 1
    END FOR

    PRINT "File successfully allocated from block " + start_block + " to " +
(start_block + length - 1)
ELSE
    PRINT "Allocation failed: Blocks requested are not free or invalid."
END IF

PRINT "Do you want to allocate another file? (1-Yes, 0-No):"
READ choice
UNTIL choice == 0
END

```

CODE:

```
#include <stdio.h>

int main() {

    int disk[100] = {0};

    int total_blocks,
    start_block, length,
    choice;

    printf("Enter total
blocks on disk: ");

    scanf("%d",
&total_blocks);

    do {

        printf("\nEnter
starting block address:
");

        scanf("%d",
&start_block);

        printf("Enter length
```

of file (number of
blocks): ");

scanf("%d",
&length);

int is_allocated = 1;

if (start_block +
length > total_blocks ||
start_block < 0) {

printf("Error:
Request exceeds total
disk allocation
bounds.\n");

is_allocated = 0;

} else {

for (int i =
start_block; i <
start_block + length;
i++) {

```
        if (disk[i] ==  
1) {  
  
        is_allocated  
= 0;  
  
        break;  
  
    }  
  
    }  
  
}
```

```
if (is_allocated) {  
  
    for (int i =  
start_block; i <  
start_block + length;  
i++) {  
  
        disk[i] = 1;  
  
    }  
  
    printf("File  
allocated contiguously  
from block %d to  
%d.\n", start_block,
```

```
start_block + length -  
1);
```

```
    } else if  
(start_block + length <=  
total_blocks &&  
start_block >= 0) {
```

```
  
printf("Allocation  
failed: Contiguous  
blocks are already  
occupied.\n");  
  
    }
```

```
  
    printf("Do you  
want to add another file?  
(1 for Yes / 0 for No):  
");
```

```
    scanf("%d",  
&choice);
```

```
    } while (choice == 1);
```

```
return 0;
```

```
}
```